CS321M: AI Measurement Science

Lecture 4 · Arc 1.3

Probabilistic Models III: Factor Models and Efficient Evaluation

Sanmi Koyejo

Stanford University · Department of Computer Science

What you will be able to do after today

Textbook reference: Chapter 4, Efficient Measurement

1. **Define** the K-factor logistic model and show that Rasch is the $K = 1$ special case
2. **Explain** why the response matrix has low-rank structure and how this enables matrix completion
3. **Write** the masked likelihood for sparse response matrices and identify when it is valid
4. **Distinguish** entry-wise, row, and column holdout splits and what each tests
5. **State** the cold-start problem and explain why standard matrix completion fails
6. **Describe** the three stages of Prediction-Guided Evaluation (PGE) and what each stage contributes
7. **Interpret** AUC results across evaluation splits and connect them to practical evaluation cost
8. **Explain** why semantic similarity does not imply behavioral similarity, and what this means for cold-start prediction

Part I

Factor Models

Beyond a Single Latent Dimension

Is one number enough to describe a model's ability?

The Rasch assumption

A single scalar θ_i captures everything about model i's capability. Given θ_i, performance on any item is fully determined by item difficulty β_j.

This is a strong claim. It says: knowing how well a model does on physics problems tells you exactly how well it does on law problems, up to item difficulty.

Evidence against it

From Lecture 2: model rankings on MMLU-Pro physics items differ from rankings on MMLU-Pro business items. That is evidence of **multidimensionality**: the thermometer test fails.

A frontier coding model may score high on STEM items and low on legal reasoning. A single θ_i cannot capture both.

The K-factor logistic model replaces the scalar θ_i with a K-dimensional ability vector $U_i \in \mathbb{R}^K$.

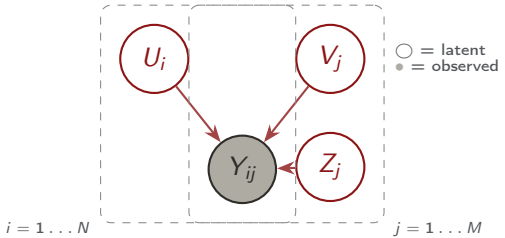
The K-Factor Logistic Model

K-Factor Logistic Model

$$P(Y_{ij} = 1 \mid U_i, V_j, Z_j) = \sigma\big(U_i^\top V_j + Z_j\big)$$

$U_i \in \mathbb{R}^K$: **ability vector** of model i
$V_j \in \mathbb{R}^K$: **loading vector** of item j
(which dimensions does this item tap?)
$Z_j \in \mathbb{R}$: **difficulty intercept** of item j
$\sigma(\cdot)$: logistic sigmoid function

Intuition. $U_i^\top V_j$ is large when model i's ability profile aligns with what item j demands. A model with high ability on dimension 1 scores well on items with high loading on dimension 1.



Rasch is a $K = 1$ special case

Setting $K = 1$

Let $K = 1$, $U_i = \theta_i \in \mathbb{R}$, $V_j = 1$ for all j, and $Z_j = -\beta_j$.

Then:

$$U_i^\top V_j + Z_j = \theta_i \cdot 1 - \beta_j = \theta_i - \beta_j$$

$$P(Y_{ij} = 1) = \sigma(\theta_i - \beta_j) \quad \checkmark$$

This is exactly the Rasch model from Lecture 2.

Model	Logit	Rank	Params
Rasch	$\theta_i - \beta_j$	2	$N + M$
2PL	$a_j(\theta_i - \beta_j)$	2	$N + 2M$
K-factor	$U_i^\top V_j + Z_j$	$K + 1$	$(N + M)K + M$

Rank refers to the logit matrix $\in \mathbb{R}^{N \times M}$.

Rasch and 2PL are low-rank models with $K = 1$. The K-factor model is a generalization that allows each model to have a **profile** of abilities, not just a single score.

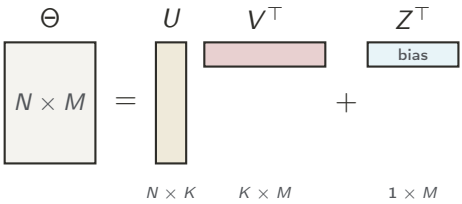
The low-rank matrix view

Logit Matrix Decomposition

$$\Theta = UV^\top + 1Z^\top \in \mathbb{R}^{N \times M}$$

$U \in \mathbb{R}^{N \times K}$: ability matrix
$V \in \mathbb{R}^{M \times K}$: loading matrix
$Z \in \mathbb{R}^{M}$: difficulty vector

$\text{rank}(\Theta) \leq K + 1$



Typical values: $K \in \{1, 2, 4\}$. Cross-validate on held-out AUC.

Why does this matter?

The full logit matrix has $N \times M$ entries. The factor model says it is determined by only $(N + M)K + M$ parameters. For $N = 172$, $M = 21{,}176$, $K = 4$: this is 85,580 parameters instead of 3,642,272.

Strong compression. But is it realistic?

Estimation follows the same logic as Lecture 3

Gradients for the Factor Model

$$\frac{\partial \ell}{\partial U_i} = \sum_{j=1}^{M} (Y_{ij} - P_{ij})\, V_j$$

$$\frac{\partial \ell}{\partial V_j} = \sum_{i=1}^{N} (Y_{ij} - P_{ij})\, U_i$$

Residuals $Y_{ij} - P_{ij}$ now weight the *vector* V_j (or U_i), not just a scalar.

Same intuition as Lecture 3: if model i gets more items right than predicted, its ability vector U_i shifts in the direction of the loading vectors V_j of those items.

Practical defaults

- **Optimizer**: L-BFGS or Adam
- **Regularization**: ℓ_2 on U and V (MAP with Gaussian priors)
- **Rank**: choose K by cross-validating on held-out AUC
- **Identifiability**: same shift ambiguity as Rasch; fix with sum-to-zero or a prior

Factor rotation (Varimax, Promax) can make factors more interpretable after fitting, but does not change predictions. We will not cover rotation here.

△ Show of Hands #1

A Rasch model is fit to MMLU-Pro. Rankings on physics items differ from rankings on business items.

What does this imply?

The Rasch model fits well: item difficulty explains the gap

The Rasch model may not fit: a single θ_i cannot explain both

Cannot conclude anything without seeing the data

The second. Rasch's specific objectivity demands that rankings be stable across item subsets. Rankings that change across domains are evidence of multidimensionality. A $K > 1$ factor model can accommodate this; Rasch cannot.

Part II

Sparse Response Matrices

The Data Reality of AI Evaluation

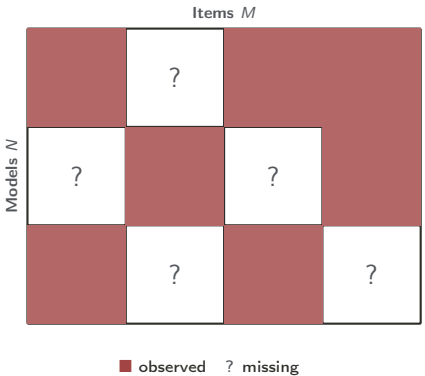
The response matrix is mostly missing

Recall the evaluation datasets from Lecture 2:

Dataset	N	M	Fill rate
Open LLM LB	4,416	21,176	$< 5\%$
HELM Suite	172	217,268	$< 10\%$
LM Arena	179	211,728	sparse

Three sources of sparsity:

- **Cost**: each API call costs money; budgets limit coverage
- **Fragmentation**: different groups test different item subsets
- **Timing**: models are released sequentially; old benchmarks are not rerun



Typical fill rate: 1–20%

Estimation with missing data: the masked likelihood

Observation Mask

Masked Log-Likelihood: Let $\Omega \subseteq \{1,\dots,N\} \times \{1,\dots,M\}$ be the set of observed entries.

$$\ell(\theta,\beta) = \sum_{(i,j)\in\Omega} \Big[Y_{ij}(\theta_i - \beta_j) - \log\big(1+e^{\theta_i-\beta_j}\big) \Big]$$

Sum only over **observed** entries; gradients are restricted to Ω as well.

When is this valid? Provided missingness does not depend on the unobserved response value itself. This fails when developers selectively report only favorable benchmarks, biasing estimates.

Three evaluation splits

Entry-wise: random held-out entries
Tests: can the model interpolate within the observed data?

Row holdout: entire models held out
Tests: can the model predict for new, unseen models?

Column holdout: entire items held out
Tests: can the model predict for new, unseen benchmarks?

Each split tests a different generalization capability. Row and column holdout require a few samples for *calibration*.

Part III

Low-Rank Matrix Completion

Filling in the Gaps

Low-rank factorization as matrix completion

Logistic Matrix Completion

$$\min_{U,V,Z} -\sum_{(i,j)\in\Omega} \log P(Y_{ij} \mid U_i, V_j, Z_j)$$

where $P_{ij} = \sigma(U_i^\top V_j + Z_j)$

Minimize masked binary cross-entropy subject to the low-rank structure.

The fitted model predicts $\hat{P}_{ij}$ for *any* model-item pair, including those never observed. This is the key benefit over accuracy-based ranking.

This is the same idea as collaborative filtering for recommendation systems, applied to binary correctness data.

What matrix completion gives you

- **Imputed rankings**: predict which model would win on a benchmark no model has run
- **Efficient evaluation**: run only a sparse subset of model-item pairs; recover the rest
- **Uncertainty**: standard errors on imputed entries from the fitted model

Requires low-rank assumption to hold, at least approximately. Checked via held-out AUC.

Empirical results: low-rank models outperform Rasch

Held-out AUC across evaluation splits on HELM (172 models, 217,268 items). Higher is better.

Model	Parameters	Entry-wise	Row holdout	Col. holdout
Mean imputation	0	0.500	0.500	0.500
Rasch ($K = 1$)	$N + M$	0.780	0.740	0.720
Factor ($K = 2$)	$2(N + M) + M$	0.820	0.770	0.745
Factor ($K = 4$)	$4(N + M) + M$	0.845	0.785	0.750
Factor ($K = 8$)	$8(N + M) + M$	**0.860**	0.780	0.740

$K = 8$: overfitting

Entry-wise AUC keeps rising with K, but row and column holdout *degrade* beyond $K = 4$. More factors fit the observed data better but generalize worse to unseen entities.

$K = 4$ is a good tradeoff. It captures the main capability dimensions without overfitting to the observed response pattern.

A factor model with $K = 4$ achieves AUC $= 0.845$ on entry-wise held-out entries.

Does this mean it can predict performance for a brand-new model with zero observed responses?

Yes: the fitted model generalizes to new rows

No: entry-wise AUC only tests interpolation

Partially: it helps but is not sufficient

The second. Entry-wise AUC holds items and models fixed; it measures interpolation. A new model has no observed entries, so there is nothing to interpolate from. Row holdout AUC is the right metric, and it requires additional machinery: the cold-start solution we cover next.

Part IV

Cold-Start and Prediction-Guided Evaluation

Predictive Evaluation Without Calibration

The cold-start problem

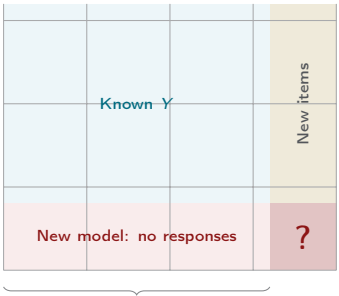
Cold-Start

A **new model** has zero observed responses. Standard matrix completion has nothing to fit: no row entries exist.

A **new benchmark** has no model evaluated on it yet. No column entries exist.

In both cases, the response matrix provides no signal. We need *side information*.

Why it matters. New frontier models are released weekly. Recalibrating the full benchmark each time is prohibitively expensive. At $0.01 per API call, running 200 models on 20,000 items costs $40,000 per calibration round.



The unknown cell: no calibration data for either entity.

Side information bridges the cold-start gap

Model side: metadata F_i

- Parameter count (log scale)
- Architecture family (GPT, Llama, ...)
- Training data size
- Release date
- Quantization level, context length

Goal: learn $f_U : F_i \mapsto \hat{U}_i$: predict the ability vector from metadata.

Item side: text embedding E_j

- Sentence embedding of the question text
- Benchmark category, subject domain
- Question length, answer format

Goal: learn $f_V : E_j \mapsto (\hat{V}_j, \hat{Z}_j)$: predict item parameters from question content.

If these mappings are learnable, we can predict $\hat{P}_{ij} = \sigma(\hat{U}_i^\top \hat{V}_j + \hat{Z}_j)$ for any model-item pair without running a single query.

A surprising finding: semantic similarity does not imply behavioral similarity

The natural baseline

If two questions have nearly identical text embeddings, intuition says models should respond similarly to both. If this were true, a simple nearest-neighbor predictor would solve the item cold-start problem.

Empirical finding (Truong et al., 2025)

For item pairs with cosine similarity > 0.99, the behavioral correlation (across model responses) spans the **entire range** $[-1, +1]$.

Semantically near-identical questions can elicit completely different response patterns from the same set of models.

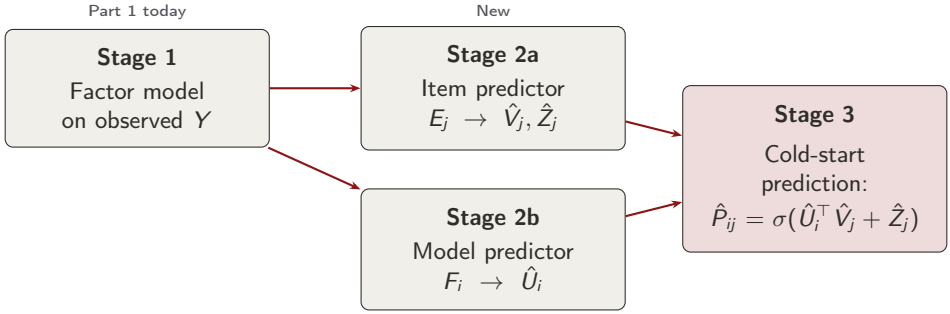
Why does this happen?

Two questions can ask "the same thing" semantically but differ in:

- Reasoning steps required
- Format (multiple choice vs. open)
- Whether the answer is in training data
- Surface-level cues that trigger or suppress retrieval

Consequence: we cannot use text similarity alone. We need a model that *learns* the semantic-to-behavioral mapping from calibration data.

Prediction-Guided Evaluation: three-stage pipeline



Stage 1 extracts behavioral structure from observed data. Stages 2a–2b learn to map side information to that structure. Stage 3 combines them to predict any model-item pair.

Stage 1: Learn the factor model from observed responses

What we fit

Joint maximum likelihood over observed entries Ω:

$$\max_{U,V,Z} \sum_{(i,j)\in\Omega} \log P(Y_{ij} \mid U_i, V_j, Z_j)$$

with ℓ_2 regularization on U and V.

Output: $U \in \mathbb{R}^{N\times K}$, $V \in \mathbb{R}^{M\times K}$, $Z \in \mathbb{R}^{M}$.

Typical: $K = 4$, chosen by held-out AUC.

What the factors capture

The rows of U are ability profiles. After fitting, we can inspect which items load highly on each factor:

- Factor 1: strong loading on STEM items
- Factor 2: strong loading on legal/social items
- Factor 3: strong loading on code tasks
- ...

Labels are post-hoc; the model discovers structure from response patterns.

Stage 2a: Item-side predictor

Neural Predictor f_θ

$$[\hat{V}_j,\ \hat{Z}_j] = f_\theta(E_j)$$

$E_j \in \mathbb{R}^d$: sentence embedding of item j's text

f_θ: MLP with ReLU activations

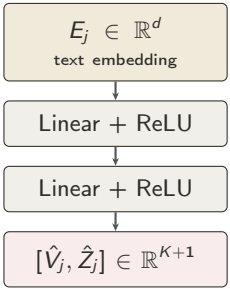
Output: $K + 1$ values ($\hat{V}_j \in \mathbb{R}^K$, $\hat{Z}_j \in \mathbb{R}$)

Training

Freeze the learned U from Stage 1. Minimize the masked binary cross-entropy:

$$\mathcal{L} = -\sum_{(i,j)\in\Omega} \log P(Y_{ij} \mid U_i, \hat{V}_j, \hat{Z}_j)$$

The network learns the mapping from question content to behavioral parameters.



The MLP must learn that questions requiring more reasoning steps are harder, even if their embeddings are similar.

Stage 2b: Model-side predictor

Linear Predictor f_ϕ

$$\hat{U}_i = f_\phi(F_i) = F_i W_U$$

$F_i \in \mathbb{R}^{d_F}$: model metadata features

W_U: learned weight matrix

A **linear** model is often good enough.

Training: ℓ_1 regression from metadata to the Stage 1 ability estimates:

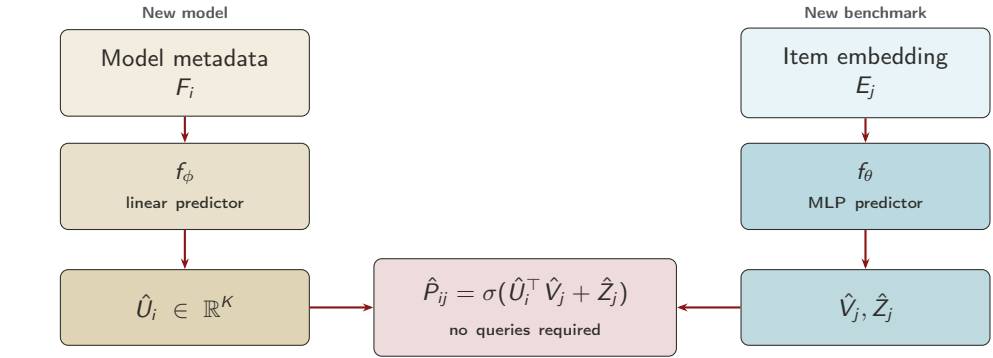
$$\mathcal{L} = \|f_\phi(F) - U\|_1$$

Why linear is enough: Model ability is largely predictable from parameter count and architecture. Larger models are generally more capable. The main structure is monotone and roughly log-linear in scale, which a linear model captures well.

What the predictor learns does not see any benchmark responses

- Models with more parameters have higher ability on most factors
- Architecture family matters: models in the same family cluster together
- Release date is informative: newer models tend to score higher

Stage 3: Cold-start prediction



The predicted probability is the full pipeline output: Stage 1 factors multiplied through Stage 2 side-information predictors. Cost is $O(N + M)$ forward passes, not $O(NM)$ API calls.

PGE results: cold-start prediction across splits

AUC on held-out entries, HELM Suite. Baseline is full matrix completion (Stage 1 only).

Method	Entry-wise	Row holdout	Col. holdout
Factor ($K = 4$, no side info)	0.845	—	—
PGE: item predictor only	0.840	0.680	**0.804**
PGE: model predictor only	0.840	**0.848**	0.720
PGE: full (both predictors)	0.840	0.841	0.800

Row holdout

AUC $\approx$ 0.85 for new models using metadata alone. Model metadata (parameter count, family) is highly predictive of ability profiles. No benchmark responses needed.

Column holdout

AUC $\approx$ 0.80 for new items using text embeddings. Despite the semantic/behavioral mismatch, the MLP learns enough of the mapping to substantially outperform chance.

What PGE gives you that accuracy and matrix completion do not

1. Evaluation efficiency

$O(N + M)$ predictions instead of $O(NM)$ queries. For 4,000 models and 20,000 items at \$0.01 per call, this is the difference between \$800 million and \$60,000.

2. Coverage without re-evaluation

A new model can get an estimated ability profile on any benchmark in the calibrated item pool, without running any queries.

3. Style-invariant measurement

PGE predicts correctness from latent factors. It does not score response style, verbosity, or formatting. This separates *what a model knows* from *how it presents answers*.

4. Multidimensional profiles

Each model gets $\hat{U}_i \in \mathbb{R}^K$: a vector ability profile, not a scalar rank. This reveals strengths and weaknesses across capability dimensions.

PGE is not a replacement for all evaluation. It complements direct evaluation by extending coverage to model-item pairs that would be prohibitively expensive to observe.

△ Show of Hands #3

Two items have cosine similarity $= 0.99$ between their text embeddings.

What can you conclude about their behavioral correlation (how similarly models respond to them)?

It will also be close to 1: similar text, similar responses

It could be anything: semantic and behavioral similarity are weakly related

It depends on the embedding model used

The second. Truong et al. (2025) show that for item pairs with cosine similarity > 0.99, behavioral correlation spans $[-1, +1]$. The MLP predictor is needed precisely because this mapping is nonlinear and must be *learned* from calibration data.

Factor models are one choice: any model of P_{ij} fits the same framework

What the PGE framework actually requires

The three-stage pipeline needs a differentiable model that takes model and item representations and produces $\hat{P}_{ij} \in [0, 1]$. The factor model is one choice, not the only one.

Can use any model that:

- Produces calibrated probabilities
- Has learnable parameters for items and models
- Supports gradient-based training on Ω

Deep learning alternatives

- **Neural IRT**: replace $\sigma(U_i^\top V_j + Z_j)$ with a deep network that learns a nonlinear interaction
- **Transformer encoders**: embed model identity and item text jointly; predict correctness end-to-end
- **Graph networks**: treat the response matrix as a bipartite graph; propagate information across models and items

The tradeoff is interpretability vs. capacity. Factor models expose meaningful parameters ($\hat{\theta}_i$, $\hat{\beta}_j$) and connect to measurement theory. Deep models may predict better but give up that structure.

Summary

Topic	Key result	Practical takeaway
Factor models	K-dim. ability vector U_i; Rasch is $K = 1$	Use $K > 1$ when rankings shift across domains
Low-rank structure	$\Theta = UV^\top + 1Z^\top$, rank $\leq K + 1$	$(N+M)K$ params instead of NM
Masked likelihood	Sum over Ω only	Valid under MAR; check split AUC
Matrix completion	Fill Y via factor model	$K = 4$ balances fit and generalization
Cold-start	No observed rows or columns	Side information required
Semantic $\neq$ behavioral	Cos. sim. > 0.99 but any behavioral corr.	Must learn the mapping, not look it up
PGE	Three-stage pipeline	AUC > 0.80 on row and col. holdout

A lab releases a new model today.
They publish its parameter count and architecture.
They have not run it on any benchmarks yet.

Using PGE, what can you produce immediately?

Nothing: you need at least some responses to estimate ability

An estimated ability vector from the model metadata predictor

A full ranking, but only on items already in the calibration pool

The second and third are both correct. Stage 2b maps metadata to $\hat{U}_i$ directly. Combining with Stage 2a item predictions gives $\hat{P}_{ij}$ for any calibrated item, producing a full predicted ranking. No queries needed.

What you should now be able to do

Checking against today's learning outcomes:

- ✓ **Define** the K-factor logistic model and connect it to Rasch
- ✓ **Explain** low-rank structure and matrix completion
- ✓ **Write** the masked likelihood and state when it is valid
- ✓ **Distinguish** entry-wise, row, and column holdout splits
- ✓ **State** the cold-start problem and explain why it is hard
- ✓ **Describe** the three PGE stages and each stage's contribution
- ✓ **Interpret** AUC results across splits
- ✓ **Explain** why semantic similarity does not imply behavioral similarity

If any of these feel shaky, review Textbook Chapter 4 (§4.1–4.2) and Truong et al. (2025). The self-quiz on the next slides will help you check.

Self-Quiz 1: Factor Models and Low-Rank Structure

Q: (a) Write the K-factor logistic model for $K = 2$. Identify all parameters.
(b) Show that setting $K = 1$, $V_j = 1$, $Z_j = -\beta_j$ recovers the Rasch model.
(c) For $N = 100$ models and $M = 5{,}000$ items with $K = 4$: how many parameters does the factor model have, versus the unconstrained logit matrix? What is the compression ratio?
(d) You fit a $K = 4$ factor model on MMLU-Pro and find that entry-wise AUC is 0.85, but row holdout AUC is only 0.61. What does this suggest?

Answers:
(a) $P(Y_{ij} = 1) = \sigma(U_i^\top V_j + Z_j)$, $U_i \in \mathbb{R}^2$, $V_j \in \mathbb{R}^2$, $Z_j \in \mathbb{R}$.
(b) $U_i^\top V_j + Z_j = \theta_i \cdot 1 - \beta_j = \theta_i - \beta_j$. Logistic of this is the Rasch model. ✓
(c) Factor: $(100 + 5{,}000) \times 4 + 5{,}000 = 25{,}400$ params. Unconstrained: $100 \times 5{,}000 = 500{,}000$. Compression: $\approx 20\times$.
(d) The model fits observed data well (interpolation) but does not generalize to new models (extrapolation). The learned U cannot be predicted from nothing. This is the cold-start problem: Stage 2b (model metadata predictor) is needed to recover row holdout AUC.

Self-Quiz 2: Cold-Start and PGE

Q: (a) A new benchmark is released with 1,000 items. No model has been run on it yet. Which PGE stage is responsible for predicting item parameters, and what does it use as input?

(b) Two items have cosine similarity 0.98 between their text embeddings. A colleague claims their item parameters $(\hat{V}_j, \hat{Z}_j)$ must also be similar. Is this claim reliable? Why?

(c) PGE achieves row holdout AUC of 0.85 using only model metadata. What would AUC $=$ 0.50 on this split mean, and what would AUC $= 1.0$ mean?

Answers:

(a) Stage 2a: the item-side MLP predictor $f_\theta(E_j) \to (\hat{V}_j, \hat{Z}_j)$. Input is the sentence embedding E_j of each item's text.

(b) Not reliable. Truong et al. (2025) show that semantic similarity near 1.0 is consistent with any behavioral correlation. The MLP is specifically trained to learn this nonlinear mapping from calibration data.

(c) AUC $= 0.50$: model metadata is uninformative; predictions are no better than random ranking. AUC $= 1.0$: metadata perfectly predicts ability ordering relative to all benchmark items — essentially free evaluation from public model specs alone.

Reference: Truong, S., Tu, Y., Liang, P., Li, B., Koyejo, S. (2025). *Reliable and Efficient Amortized Model-based Evaluation.* ICLR 2025. arXiv:2503.13335.

Next lecture: Sample-Efficient Measurement Active Learning

Textbook: aimslab.stanford.edu/textbook

Course: cs321m.stanford.edu